License Plate Detection System using Jetson Nano

Concepts and Application

Gautam Ramesh, 7026787
Siddhanth Sharma, 7027189
Shubhankar Dumka, 7027187

5. Januar 2026

1. Introduction
2. Description of Application
3. Challenges and Solutions
4. System Setup
5. Results
6. Results
7. References

Introduction

Introduction I

- The project focuses on designing an automatic car license plate detection system using the Nvidia Jetson Nano.
- License plates are detected using ML models (CNN) deployed on embedded devices. [Tan+22] [JCQ23; Cor21].
- Application: Places with high vehicle throughput such as airport departures, where manual verification is inefficient and error-prone.

Description of Application

Description of Application I

- **Target deployments:** Airport curbside (drop-off time control) and parking facilities (paid stay management).
- **Core idea:** detect the license plate at **entry** and **exit** using a camera + Jetson Nano, and use timestamps to compute dwell time.
- **Edge-first operation:** Recognition and decision making run locally enabling low latency. Reduced dependence on centralized servers.
- **Operator-specific configuration (before delivery):** the airport/parking authority provides billing requirements (tariff, grace period, rounding rules); we configure the system accordingly and deploy it.
- **System output:** validated plate ID, entry/exit events, and a fee/permission decision shown on a display.

Description of Application II

- **Entry flow:** camera captures vehicle → YOLO detects license plate → OCR reads text → store `(plate_id, entry_timestamp)` in database.
- **Exit flow:** detect plate again → retrieve `entry_timestamp` → compute dwell time `(exit - entry)`.
- **Billing logic (configured per operator):** apply provided parameters (grace minutes, €/hour tariff, rounding) → compute amount due.
- **Decision & display:** show "free exit" or "payment required" with the calculated amount due.
- **Logging:** write `exit_timestamp` and fee outcome for auditing and analytics.

Description of Application III

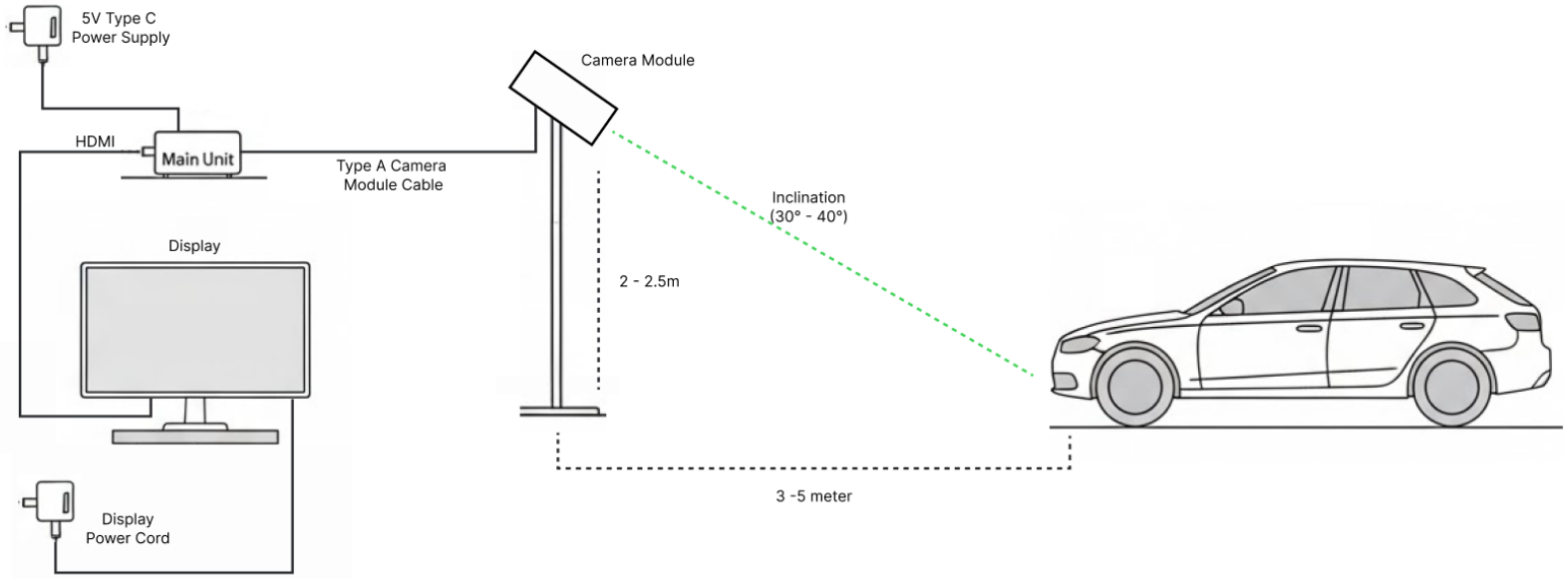

Abbildung: License Plate Detection System.

Challenges and Solutions

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Challenge 1: Storage Constraints

The Problem	The Solution
Hardware Limitation (Production Module) • **No SD Slot:** Our unit is the *Jetson Nano Production Module* (P3448), which lacks an SD card slot by design. • **Space Lock:** The fixed 16GB internal storage (eMMC) is too small for our AI environment (CUDA + OS).	**External Boot Architecture** • **SSD Integration:** We engineered the system to bypass the internal memory and boot directly from an external 500GB SSD via USB 3.0. • **Result:** Solved the space issue and increased data read speeds by 5x (400 MB/s).

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Challenge 2: System Memory Exhaustion

The Problem	The Engineered Solution
Conflict: Unified Memory Bottleneck The Jetson Nano shares a single 4GB RAM pool between the CPU and GPU. • **OS Overhead:** Ubuntu 18.04 consumes ≈ 1.2 GB. • **AI Load:** YOLOv5 + EasyOCR required ≈ 3.3 GB during initialization. *Impact:* Demand (4.5 GB) exceeded physical limits. The Linux kernel invoked the OOM Killer and terminated the process instantly.	**Solution: Hybrid Memory Architecture**: • **Virtual Expansion:** Configured a **6GB Swap Partition** on an external SSD to offload inactive OS pages, keeping physical RAM free for inference. • **Compute Isolation:** Forced OCR to run on the **CPU** (`gpu=False`). This reserves the limited GPU VRAM entirely for YOLO detection.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Challenge 3: Legacy Software Ecosystem

The Problem	The Engineered Solution
Conflict: Dependency Conflicts The Jetson Nano relies on JetPack 4.6 (Ubuntu 18.04), which locks the environment to **Python 3.6**. • **Incompatibility:** State-of-the-art libraries (Ultralytics YOLOv8, Pandas 2.0) require Python 3.8+ and modern C++ compilers, causing installation failures. • **Deprecated APIs:** The `Pillow` image library recently removed attributes (like `ANTIALIAS`) used by older Jetson inference tools, causing runtime crashes.	**Solution: Cross-Compilation Workflow** We bypassed the OS limitations using an external pipeline: • **TensorRT Export:** Instead of running PyTorch natively, we trained the model on a PC, exported it to **ONNX**, and cross-compiled it into a **.engine** binary specifically optimized for the Nano's Maxwell GPU. • **Monkey Patching:** We developed a custom initialization script that injects code at runtime to manually map old API calls to their new locations, preventing crashes in legacy libraries.

Challenge 4: Remote Development Breakdown

The Problem	The Engineered Solution
Conflict: glibc Version Lock In 2024, Microsoft updated the VS Code Remote Server to require **glibc 2.28** or higher. • **The Lockout:** The Jetson Nano's OS is pinned to **glibc 2.27**. • **Impact:** Standard SSH connections failed with OS Not Supported errors, blocking all direct code editing and file transfer capabilities on the device.	**Solution: SSH-FS Protocol Switch** We architected a workaround that avoids running binary agents on the Jetson: • **SFTP Mounting:** We switched to the **SSH FS** extension, which mounts the Jetson's external SSD (/media/myssd) as a local network drive on the development laptop. • **Benefit:** This protocol relies purely on standard SFTP, which does not check the glibc version, allowing full real-time coding access despite the OS obsolescence.

System Setup

Setup with SSH connection

- **Unit:** NVIDIA Jetson Nano.
- **Camera:** Microsoft LifeCam for video input.
- **Control:** Laptop via SSH for debugging.
- **UI:** External monitor showing Flask Dashboard.

The hardware setup

Results

System Demonstration Video

Click image to launch video player.

Results

Model Performance

- **YOLO Detection:** 100% accurate. The model locks onto the license plate perfectly in every frame, even with movement.
- **OCR Limitations:** Text recognition is inconsistent. While detection is perfect, the OCR sometimes misinterprets characters (e.g., swapping order due to angle).
- **Latency:** The video feed pauses briefly during recognition, confirming that the heavy OCR processing is happening locally on the Jetson Nano.

System Logic Verification

- **Handling Misreads:** When the OCR reads a wrong number (e.g., `66ZLA`), the database logic correctly treats it as a New Vehicle entry.
- **Successful Billing:** Despite the OCR challenge, when the text *is* read correctly, the billing math and timestamp updates work perfectly (**€99.20**).

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Thanks a lot

for your attention

References

References I

[Cor21] A. Cortés. "Edge AI for Real-Time Computer Vision Applications". In: *Proceedings of the International Conference on Embedded Systems*. 2021, S. 88–95.

[JCQ23] Glenn Jocher, Ayush Chaurasia und Jing Qiu. *Ultralytics YOLOv8*. 2023. URL: https://github.com/ultralytics/ultralytics.

[Nat24] National Airports Council. *Vehicle Access Management Challenges in Modern Airports*. Techn. Ber. National Airports Council, 2024.

[Tan+22] Y. Tan u. a. "A Survey on Automated Number Plate Recognition Systems". In: *IEEE Access* 10 (2022), S. 12345–12360. DOI: 10.1109/ACCESS.2022.XXXXXXX.